MIESC

Multi-layer Intelligent Evaluation for Smart Contracts

Malda (malda-lending) — Focused Triage

malda-protocol/malda-lending @ ab4fa9b2 · Sherlock contest scope · 195 findings, 0 exploitable Highs · judged H-1 sits outside Slither's lens

PUBLIC

Malda (malda-lending) — MIESC audit triage memo

Target: `malda-protocol/malda-lending` @ `ab4fa9b2` · Sherlock contest scope (26 first-party `src/` files) **What it is:** A Compound-V2-style **unified-liquidity lending** protocol (`mToken` / `mErc20` markets, `Operator` comptroller, `JumpRateModelV4` interest, `MixedPriceOracle` feeds) with a **cross-chain** layer: an `mTokenGateway` / `mErc20Host` extension-vs-host market pair, a `Rebalancer` that moves market liquidity across chains through pluggable bridge adapters (`EverclearBridge`, Across), and a `Migrator` that pulls Compound-fork (Mendi) positions into Malda. A ZK coprocessor (`malda-zk-coprocessor`, Rust) proves cross-chain state — **out of scope here** (Slither is Solidity-only). Deploys on Ethereum + Linea/OP/Base. **Toolchain:** Foundry / **solc 0.8.28** (london, optimizer 200, no via_ir) + **Slither 0.11.6**, reusing forge build-info. **Scan (contest scope): 195 owned findings — 0 Critical / 2 High / 25 Medium / 80 Low / 88 Info**. **Method note:** MIESC ran Slither with its **default detector set**, deliberately *without* the project's `foundry.toml [lint]` config, and scoped to the **exact 26 files Sherlock put in-scope** (not all 73 `src/` files). 158 raw results in first-party `src/` files *outside* the contest scope were set aside (**0 of them High**); 46 dependency results under `lib/` were excluded.

> Triage layer. **Nothing here is a submission.** No PoC was built or submitted. The Sherlock audit contest on this commit **closed (judging complete; the final report is public)**, so this write-up carries no live-competition embargo; it reconciles MIESC's output against the project's own published Sherlock report (1 High + 16 Medium judged valid). Any disclosure is the user's own action.

Headline

Both scanner Highs are false positives — but the more important result is what the scanner could *not* see. The contest's one judged **High (H-1)** is a real, paid bug living in the **same** `Rebalancer` **/bridge subsystem MIESC flagged**, yet it is **invisible to Slither's detectors**: the `EverclearBridge.sendMsg` path forwards a rebalancer-supplied `bytes32 receiver` straight into an external

`everclearFeeAdapter.newIntent(...)` with **no validation that** `receiver` **is a legitimate market**, letting the privileged rebalancer EOA redirect bridged funds to itself. That is a **trust-boundary / business-logic invariant**, not a native-ETH send or a tainted-`transfer` destination, so no `arbitrary-send-*` detector models it. MIESC's two raw Highs in the same subsystem (`Rebalancer.saveEth`, `Migrator.migrateAllPositions`) are both false positives. **Seventh target in a row with 0 exploitable Highs from the static scan.** It is also the first where a real High sat squarely inside the scanned code and the static lens still could not reach it — and that boundary is the lesson.

High findings — per-site verdict (2/2 FP)

`arbitrary-send-eth` → **FP** · `Rebalancer.saveEth()` **L88**

```
function saveEth() external {
    if (!roles.isAllowedFor(msg.sender, roles.GUARDIAN_BRIDGE())) revert Rebalancer_NotAutho
    uint256 amount = address(this).balance;
    (bool success,) = saveAddress.call{value: amount}("");   // flagged
    require(success, Rebalancer_RequestNotValid());
    emit EthSaved(amount);
}
```

The raw `.call{value}` recipient is `saveAddress`, which is **declared at L43, assigned only in the constructor (L61, behind a non-zero check), and has no setter anywhere in the contract** (`rg 'saveAddress'` returns exactly the declaration, the constructor assignment, and this `.call`) — so it is **not attacker-chosen**. Slither's own `immutable-states` detector independently flags `Rebalancer.saveAddress` as "should be immutable", corroborating that it is constructor-only. The function is `GUARDIAN_BRIDGE`-gated. This is a standard **ETH-dust rescue** sweeping the contract's own balance to the fixed rescue address. Not exploitable. FP.

`reentrancy-balance` → **FP** · `Migrator.migrateAllPositions()` **L83**

Flagged for the balance delta around the redeem in step 5 (`underlyingBalanceBefore`/`After` bracketing `IMendiMarket.redeemUnderlying`, then a `>=` safety assert). The function migrates **the caller's own** Mendi (Compound-fork) positions into Malda. Every external callee is trusted and fixed: the **Mendi Comptroller/markets** (the real deployed source protocol, read via `getAssetsIn(user)`), their **configured underlying ERC20s**, and the destination **Malda market** — `_getMaldaMarket` (L193) resolves it only to a market that is **both listed in the** `Operator` **and present in the constructor-seeded** `allowedMarkets` **set** (8 addresses, no setter), so it is never attacker-chosen. Collateral is pulled with `safeTransferFrom(msg.sender, …)` and redeemed/transferred to the caller's own market; a reentrant underlying could at most re-enter the caller's *own* migration — **no cross-user drain vector**. The balance bracket is a redeem-output check, not a value-bearing state write. FP — consistent with the judged report, which flags a Migrator **slippage** issue (M-3), not reentrancy.

The bug MIESC could not see — judged H-1 (`EverclearBridge.sendMsg`)

```
function sendMsg(uint256 _extractedAmount, address _market, uint32 _dstChainId,
                 address _token, bytes memory _message, bytes memory) external payable onlyR
    IntentParams memory params = _decodeIntent(_message);
    require(params.inputAsset == _token, Everclear_TokenMismatch());   // token checked
    require(_extractedAmount >= params.amount, BaseBridge_AmountMismatch());
    // ... destination-chain membership checked ...
    everclearFeeAdapter.newIntent(
        params.destinations, params.receiver,   // <-- receiver NEVER validated
        params.inputAsset, params.outputAsset, params.amount, ...
    );
}
```

`sendMsg` validates the token, the amount and the destination chain, but **never validates** `params.receiver` (a `bytes32` decoded from the rebalancer-supplied `_message`). The privileged rebalancer EOA can set `receiver` to an address it controls on the destination chain and **redirect the bridged** `params.amount` **to itself** (fix: force `receiver == _market`). Slither's in-scope output for this file was only `unused-return` (L79) and `uninitialized-local` (L95) — **not one detector pointed at the receiver**. Why the static lens misses it: - The "send" is a `bytes32 receiver` **argument to an external** `newIntent(...)` **call** on a fee adapter, not a native-ETH transfer (`arbitrary-send-eth`) or a direct `transfer/transferFrom` with a tainted destination (`arbitrary-send-erc20`). Slither has no model that `newIntent`'s `receiver` is a value sink. - The vulnerability is a **business-logic invariant** ("a privileged role may move funds, but only to a valid market") — exactly the class of property a pattern detector cannot encode.

This is the crisp, honest boundary of MIESC's Slither layer: a paid High, inside the scanned scope, in the very subsystem that produced the scanner's two (false-positive) Highs, and still out of reach.

Other classes (characterized)

Class	Count	Verdict
`unused-return`	14	Compound-style ignored return codes (`Operator` / `mErc20Host` / oracle `latestRoundData`) — standard; characterized by detector class, not individually opened.
`reentrancy-no-eth` / `-events` / `-benign`	2 / 22 / 10	Lending CEI subclasses on trusted market/operator callees; Low/Info noise, same posture as the Migrator High.
`incorrect-equality`	5	`== 0` / sentinel guards (`mToken`, `mTokenStorage`); characterized by detector expression, not individually opened.
`uninitialized-local`	3	All benign zero-seeds, opened: `Operator:551` `uint256 sum` accumulator, `Operator:755` `AccountLiquidityLocalVars vars` (the canonical Compound liquidity-calc struct, fields written before read), `EverclearBridge:95` `bool found` loop flag.
`divide-before-multiply`	1	

Class	Count	Verdict
		`JumpRateModelV4.getSupplyRate` L163-164 — the canonical Compound `rateToPool = borrowRate * (1e18 - reserveFactor) / 1e18; utilizationRate * rateToPool / 1e18` supply-rate idiom; the mantissa truncation is intended.
`calls-loop` (22), `timestamp` (11), `naming-convention` (63), `missing-zero-check` (6), `costly-loop`, `assembly`, …	—	Noise / standard; characterized by detector class, not individually opened.

Reconciliation against the judged Sherlock report

The contest judged **1 High + 16 Mediums** valid (H-1 above; M-1…M-16). Reading the finding tables: **every judged issue is semantic/business-logic** — unchecked cross-chain `receiver` (H-1), redeem rounding direction (M-1), first-depositor borrow-rate brick (M-2), Migrator share-vs-underlying slippage (M-3), blacklist bypass on `outHere` (M-4), rebalancer destination/fee/bridge-flow bugs (M-5/6/7/9/10/12), oracle token coverage (M-14), bridge-failure fund lock (M-15), and more. **None of them is a class Slither's detectors produce.** MIESC's static scan surfaced **zero** of the judged issues and reconfirmed its own two raw Highs as false positives — a true-negative on the real bugs, not a conflict. What the baseline establishes is that the areas the detectors *did* fire on (the ETH-rescue send, the migration reentrancy bracket) are not where the real risk lived; the real risk was in invariants the static layer cannot express.

Bottom line

- **No submittable bug** from static scan + triage — **seventh** target in a row (Lombard, Doppler, Valantis STEX, YieldNest, Arcadia, v4lend, **Malda**) with all raw Highs being false positives.
- **The headline finding is a limitation, honestly stated:** a real, paid **High (H-1)** lived in the scanned scope, in the same `Rebalancer`/bridge subsystem that produced both of MIESC's false-positive Highs, yet was **invisible to Slither** because it is a trust-boundary invariant (an unchecked `receiver` forwarded to an external intent call), not a pattern any `arbitrary-send-*` detector models. This is the clearest demonstration across all targets of **where MIESC's static lens ends and semantic review must begin** — and it is exactly the case the triage layer exists to make legible.
- Honest limits: verdicts are from reading the code against the contract NatSpec and the **public Sherlock judged report**; **no PoC was built or run**. MIESC's Slither lens does not cover the cross-chain/accounting/oracle semantic classes the contest targeted.

Appendix — Evidence trail

- Normalized scan: `results.json` (owned = the 26 contest-scope `src/` files; first-party `src/` files outside the contest scope and `lib/` dependencies excluded — 0 Highs among the out-of-scope first-party results).

- Reports: `triage-focused.pdf` (focused triage); the full premium PDF is regenerable from `results.json` and is not committed.
- Toolchain: HTTPS clone @ `ab4fa9b2` + submodules → `forge build --root <repo> --use <solc-0.8.28> --build-info --skip test/script` → `slither <repo> --foundry-ignore-compile --json` (slither 0.11.6, default detectors). Build/scan were invoked from a safe working directory with the repo passed by absolute path (never `cd`-ing into the untrusted clone); that is why Slither's `filename_relative` paths carry a `malda-audit/repo/` prefix, which the normalizer strips before scope-matching.
- Target pin: `malda-protocol/malda-lending` @ `ab4fa9b2da94` (Sherlock "2025-07-malda" contest scope).
- Only **public source from the public GitHub repository** and the **public Sherlock judged report** were accessed; **static analysis only**, no live systems / RPC / credentials, no exploitation, nothing submitted anywhere.